Describing a price line with Kolmogorov-Arnold Networks: one edge versus a hidden layer

A plain-language but rigorous account of one experiment: fitting KANs to three share-price series, first with a single learnable edge and then with a hidden layer. Written for a reader with no background in computer science, statistics or higher mathematics. Every number is taken from the saved experimental output; every formula is stated in full.

1. The question

Take the daily closing price of a share over three and a half years. It is a jagged line with about 900 points. Suppose you may describe that line using only a small list of numbers, say 15, or 60, or 240, and someone else must redraw the line from your list alone. How close can the redrawn line get? And how much closer does it get each time you are allowed more numbers? That is the whole experiment. In mathematics it is called **function approximation**: finding a simple formula whose graph stays close to a complicated one.

This is deliberately **not** forecasting. Nobody is asked what the price will be tomorrow. The entire series, past and future, is on the table from the start. The earlier part of this project showed that no model we tried could forecast prices better than simply repeating yesterday's price, because day-to-day price changes are close to unpredictable. Approximation sidesteps that: it asks only how well a given kind of formula can *describe* what has already happened, which is a question about the formula, not about the market.

Why it matters. If one kind of formula describes price paths far more compactly than another, that tells us something about the formula's expressive power that forecasting experiments cannot, because forecasting results are dominated by the unpredictability of the target rather than by the model. It also tells us how 'rough' price data is, which turns out to be the deepest finding here.

2. The data

Three shares, chosen to be as different as possible: Commonwealth Bank (CBA.AX, an Australian bank, the series the original paper used), Tesla (TSLA, very volatile) and Eli Lilly (LLY, a steady upward trend). Daily closing prices from 2 January 2020 to 31 July 2023: 905 trading days for CBA, 901 for the two US shares.

Two rescalings make the three series comparable. Time is rescaled so that the first day is t = 0 and the last day is t = 1. Price is rescaled so that the lowest close in the period is 0 and the highest is 1 (this is called Min-Max scaling: p_scaled = (p - p_min) / (p_max - p_min)). After this, every series is a set of about 900 points (t_i, p_i) with both coordinates between 0 and 1. An error of 0.01 therefore means one hundredth of the share's whole price range over the period.

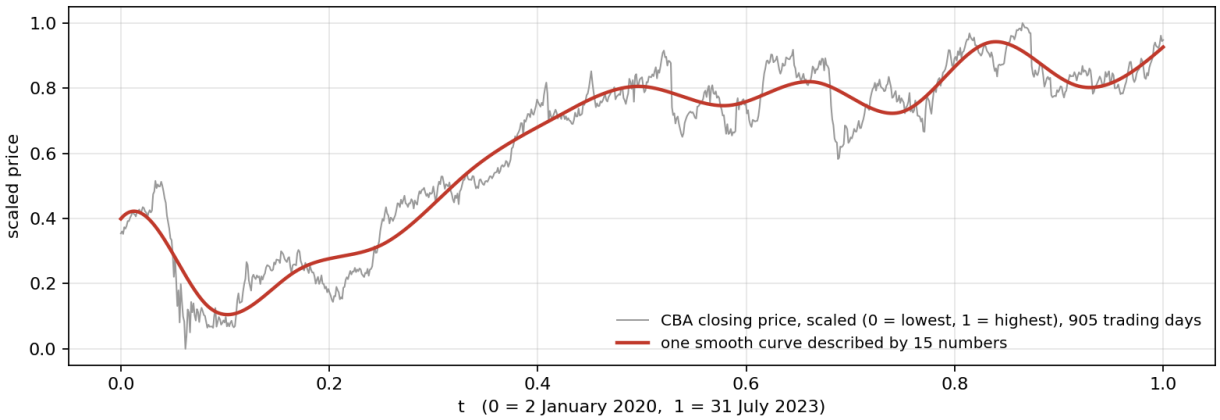


Figure 1. The CBA series after rescaling (grey), and one smooth curve that tries to follow it using only 15 numbers (red). The curve catches the broad movements and misses the fine ones. Everything in this document is about making curves like the red one, with different kinds of formula and different numbers of parameters.

3. The mathematics we need

3.1 Functions and parameters

A **function** is a rule that turns an input into an output. Here the input is a time t between 0 and 1 and the output is a price. We write f(t). A **family** of functions is a formula with blanks in it; the numbers that fill the blanks are the **parameters**. For example f(t) = a + b t + c t^2 is a family with three parameters a, b, c; choosing values for them picks one parabola out of the family. 'Describing the price line with 60 numbers' means picking the member of a 60-parameter family that sits closest to the data.

3.2 Measuring closeness: the error and the RMSE

For each day i we have the real price p_i and the curve's value f(t_i). Their difference e_i = f(t_i) - p_i is that day's **error**: positive if the curve is above the price, negative if below. One number summarises all the errors, the **root-mean-square error**:

```
RMSE = sqrt( (1/n) * sum over i of e_i^2 ) (n = number of days)
```

In words: square every error (so positives and negatives do not cancel and big errors count more), average the squares, take the square root to get back to price units. A second number, the **maximum absolute error** max |e_i|, records the single worst day. Both are reported for every fit.

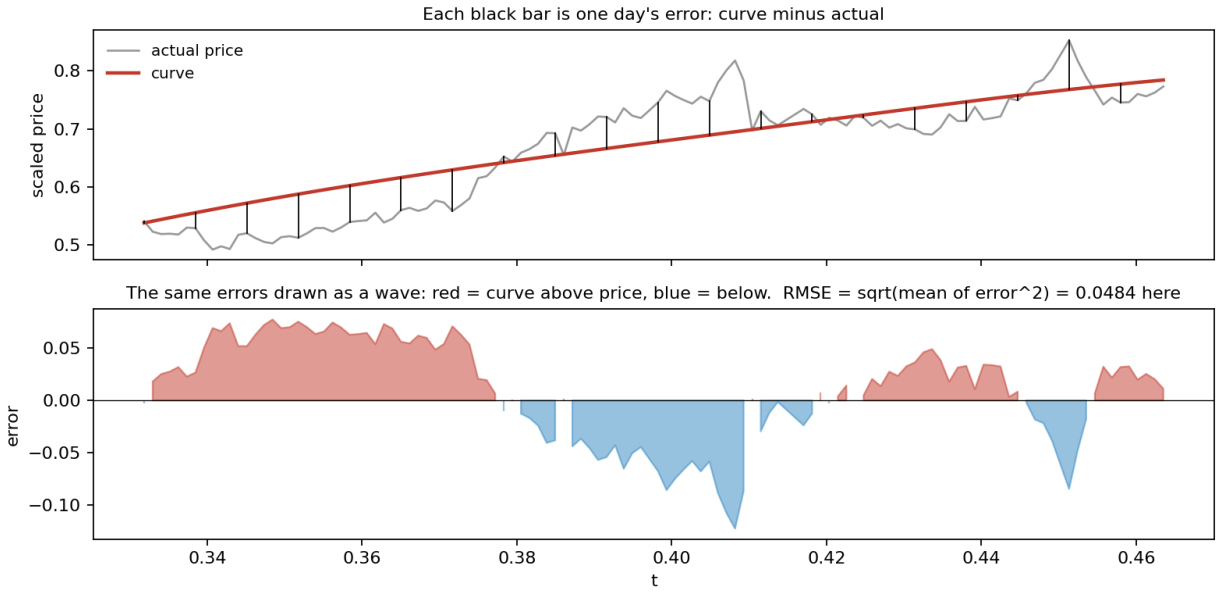


Figure 2. Top: a stretch of the price (grey) and a curve (red); each black bar is one day's error. Bottom: the same errors drawn as a wave, red where the curve is above the price, blue where below. The RMSE is the typical height of this wave. The result PDFs draw this wave for every fit, with a black envelope showing the 20-day rolling RMSE, i.e. the local typical error with the day-to-day jitter smoothed out.

3.3 'Least squares'

Among all members of a family, the one we want is the one with the smallest RMSE. Because the RMSE is built from squared errors, this is called the **least-squares fit**. For some families (splines) the least-squares fit can be computed exactly by solving a system of linear equations; for others (neural networks) it must be searched for by trial and improvement, which is what 'training' means, and the search may stop at a good-but-not-best answer.

3.4 Comparing fairly: parameter budgets

A formula with more parameters can always fit at least as well, so comparing two formulas is only meaningful at the **same number of parameters**. We therefore fixed five budgets, about 15, 30, 60, 120 and 240 parameters, set each model up to use as close to the budget as its structure allows, and recorded the exact count actually used. The plots put the true count on the horizontal axis.

3.5 Rate of improvement, and why we use logarithmic axes

The most informative quantity is not any single error but how fast the error shrinks as the parameter count N grows. For many approximation methods the error follows a **power law**, error = C * N^(-a), where a is the **rate**. Taking logarithms turns this into log(error) = log C - a * log N, a straight line whose slope is -a. So on axes where both the parameter count and the error are drawn logarithmically, a straight line means a power law and its steepness is the rate: slope -1 means doubling the parameters halves the error; slope -0.5 means doubling the parameters only cuts the error by a factor of 1.41 (the square root of 2). We fit this slope to each model's results.

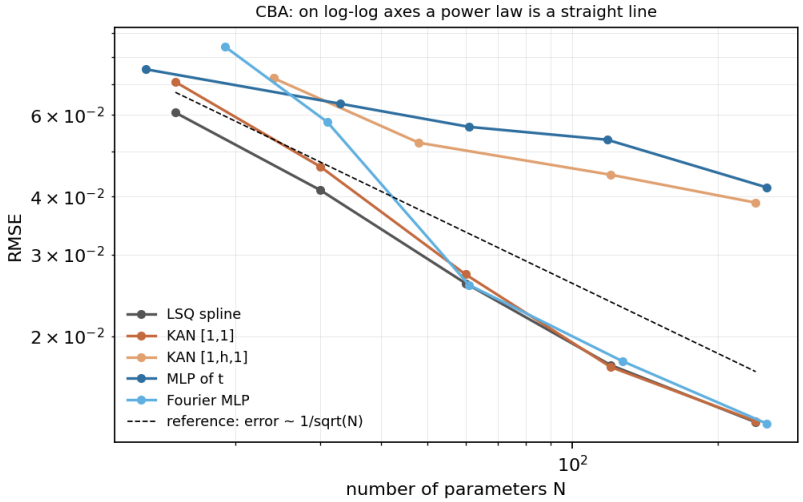


Figure 3. The CBA results on logarithmic axes. Each model's points lie roughly on a straight line; the dashed reference line has slope -0.5 (error proportional to 1 / sqrt(N)). The slope of each model is listed in the results section.

3.6 How rough is a price line? (the key background fact)

Zoom into a smooth curve and it eventually looks like a straight line. Zoom into a price line and it never does: at every magnification it is still jagged (Figure 4). Mathematicians measure this with the **Holder exponent**: a function is Holder-continuous of order alpha if |f(x) - f(y)| is at most a constant times |x - y|^alpha for all nearby x, y. A smooth curve has alpha = 1 (it is 'Lipschitz'); the path of a random walk, which is the standard model of daily prices, has alpha just below 1/2 and no higher (this is a theorem about Brownian motion; see Morters and Peres, *Brownian Motion*, Chapter 1). Classical approximation theory (de Boor, *A Practical Guide to Splines*) then says that any piecewise-smooth approximation with N pieces of a Holder-alpha function has error of order N^(-alpha). For a price path that predicts a rate of about 0.5: **doubling the parameters can only reduce the error by about 30 %, whatever the architecture**. Keep this prediction in mind; the results test it.

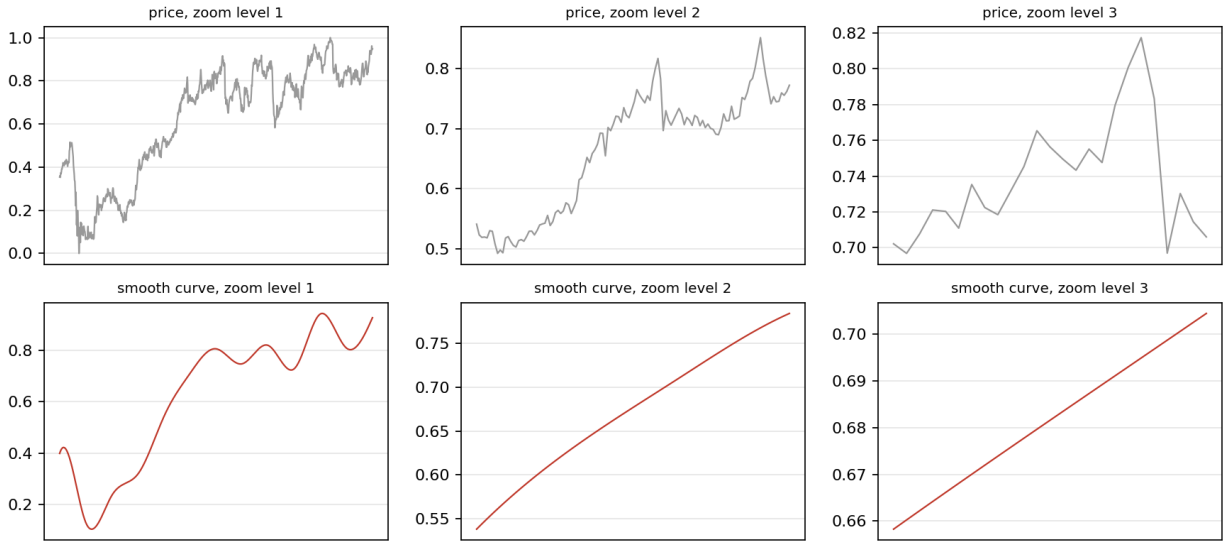


Figure 4. Top row: the CBA price line at three magnifications; it stays jagged. Bottom row: a smooth curve at the same magnifications; it straightens out. This difference is what limits every approximator in this study.

4. The two models

4.1 What a KAN is

A neural network is a diagram of nodes and edges through which a number flows from input to output. In the familiar kind (an MLP, the subject of the third document) each edge multiplies the number by a weight and each node applies a fixed squashing function. A **Kolmogorov-Arnold Network** (KAN; Liu et al., 2024, arXiv:2404.19756) turns this inside out: **each edge carries its own learnable function** phi, and nodes do nothing but add up what arrives. There are no weights and no fixed squashing functions. The name comes from a theorem of Kolmogorov and Arnold (1957) stating that any continuous function of several variables can be written exactly as sums of functions of one variable, which is what such a network computes.

In the implementation used here (pykan 0.2.8) every edge function has the form

```
phi(x) = w_b * silu(x) + w_s * s(x), silu(x) = x / (1 + e^(-x)), s(x) = sum_j
c_j B_j(x)
```

that is, a small fixed 'base' term (silu, a smooth ramp) plus a **cubic B-spline** s(x) on G intervals, exactly the object described in the splines document, with G + 3 learnable coefficients c_j and two learnable scales w_b, w_s. So a KAN edge is a spline with a ramp added. The parameters are fitted by a numerical optimiser (L-BFGS, 80 steps), not by solving linear equations, because the scales multiply the spline and make the problem non-linear.

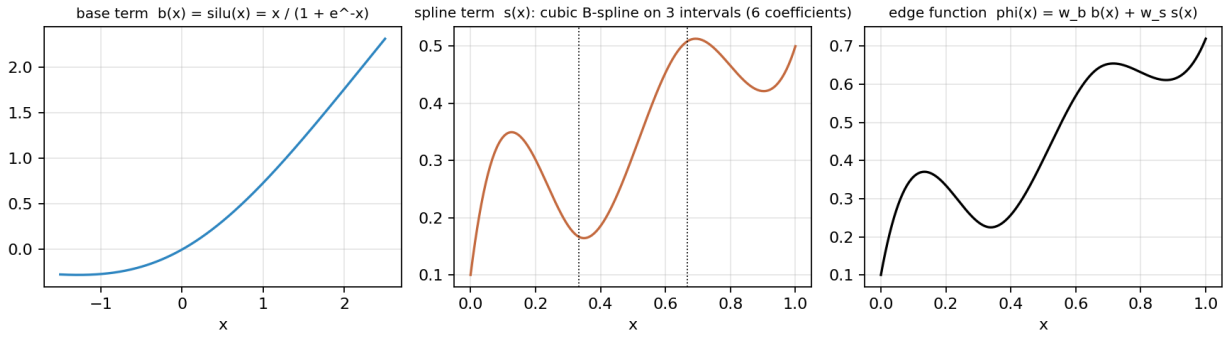


Figure 5. The three ingredients of one KAN edge: the fixed ramp silu(x) (left), a cubic B-spline on 3 intervals with its 6 coefficients (middle), and their weighted sum, the edge function phi(x) (right).

4.2 Model A: KAN [1,1], one edge

One input node (t), one output node (the price), one edge between them. The whole model is therefore a single edge function: f(t) = w_b silu(t) + w_s s(t). With the spline on G intervals this has G + 9 parameters (G + 3 coefficients, the two scales, and four fixed-structure terms pykan always includes). Budgets 15 to 240 use G = 6, 21, 51, 111, 231.

4.3 Model B: KAN [1,h,1], a hidden layer

Between input and output sits a layer of h nodes. There are h edges from t to the hidden nodes and h edges from them to the output: 2h learnable functions in all. The output is

```
f(t) = sum over k of psi_k( phi_k(t) ), k = 1 .. h
```

a sum of h *compositions*: first phi_k reshapes t, then psi_k reshapes the result. Each edge's spline has only G = 3 intervals (pykan's default), so each is coarse; the hope is that composing coarse functions creates fine detail. Parameters are about 24 per hidden node; h = 1, 1, 2, 5, 10 give 24, 24, 48, 120, 240 (the two smallest budgets cannot be met, since even h = 1 costs 24). This model has random initial values, so three seeds were run.

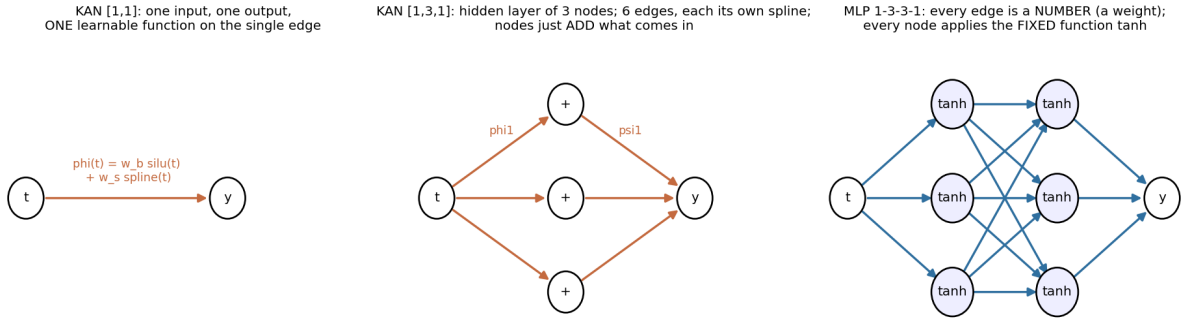


Figure 6. Left: KAN [1,1], one learnable function on one edge. Middle: KAN [1,3,1], six learnable functions and three adding nodes. Right, for contrast: an MLP, where edges are numbers and nodes apply the fixed function tanh (see the third document).

4.4 The one thing that differs

Depth. Model A puts all its parameters into the resolution of one spline (many intervals on one edge). Model B puts them into more edges with coarse splines and relies on composition. Same edge function, same optimiser, same data; only the wiring changes.

5. The experimental setup

For each of the three shares, each model and each of the five parameter budgets, the model was fitted to the whole series by least squares (exactly for splines, by numerical optimisation for the networks). Models whose fitting involves randomness (random starting values, random frequencies) were run three times with different random seeds (0, 1, 2) and the mean and spread are reported. In total 210 fits. For every fit we saved the fitted value on every day, the RMSE, the maximum error, the parameter count, the wall-clock time and the settings.

Settings per budget for the two models in this document:

budget	15	30	60	120	240
KAN [1,1]: grid G / parameters	6 / 15	21 / 30	51 / 60	111 / 120	231 / 240
KAN [1,h,1]: hidden h / parameters	1 / 24	1 / 24	2 / 48	5 / 120	10 / 240

Software: Python 3.10, NumPy/SciPy for splines, pykan 0.2.8 for KANs, PyTorch 2.2 for MLPs; single CPU. Code: approx_study.py (fits), aggregate_approx.py (tables and figures), build_pdfs.py (result PDFs).

6. Results

share	budget	KAN [1,1] RMSE	max err	params	KAN [1,h,1] RMSE	max err	params
CBA.AX	15	0.0707 +- 0.0000	0.2201	15	0.0720 +- 0.0064	0.2277	24
CBA.AX	30	0.0464 +- 0.0000	0.1804	30	0.0720 +- 0.0064	0.2277	24
CBA.AX	60	0.0272 +- 0.0000	0.0954	60	0.0523 +- 0.0025	0.1842	48
CBA.AX	120	0.0172 +- 0.0000	0.0804	120	0.0446 +- 0.0041	0.1786	120
CBA.AX	240	0.0131 +- 0.0000	0.0696	240	0.0388 +- 0.0016	0.1414	240
TSLA	15	0.0905 +- 0.0000	0.3084	15	0.0877 +- 0.0061	0.2912	24
TSLA	30	0.0495 +- 0.0000	0.1920	30	0.0877 +- 0.0061	0.2912	24
TSLA	60	0.0295 +- 0.0000	0.1393	60	0.0592 +- 0.0057	0.2247	48
TSLA	120	0.0190 +- 0.0000	0.0818	120	0.0420 +- 0.0037	0.1817	120
TSLA	240	0.0138 +- 0.0000	0.0721	240	0.0407 +- 0.0044	0.1843	240
LLY	15	0.0491 +- 0.0000	0.1440	15	0.0481 +- 0.0036	0.1472	24
LLY	30	0.0261 +- 0.0000	0.0790	30	0.0481 +- 0.0036	0.1472	24
LLY	60	0.0173 +- 0.0000	0.0635	60	0.0315 +- 0.0043	0.1001	48
LLY	120	0.0120 +- 0.0000	0.0483	120	0.0233 +- 0.0018	0.0809	120
LLY	240	0.0083 +- 0.0000	0.0342	240	0.0243 +- 0.0017	0.0774	240

Table 1. RMSE (mean +- standard deviation over three seeds where the model is random) and mean maximum error, in the share's scaled price units, with the exact parameter count. For scale, the one-day-ahead naive forecast error of the same series is 0.0201 (CBA), 0.0255 (TSLA), 0.0148 (LLY).

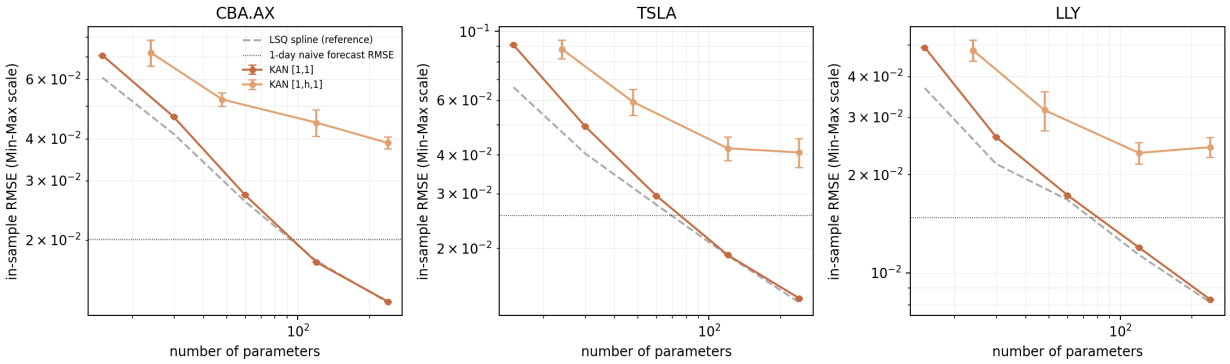


Figure 7. Error against parameter count for the two KANs on the three shares (log-log). The grey dashed line is the classical fixed-knot spline from the first document, for reference; the dotted horizontal line is the one-day naive forecast error.

Rates of improvement (slope of log RMSE against log parameters, budgets 30 to 240): KAN [1,1] -0.61, -0.62, -0.55 (mean -0.59); KAN [1,h,1] -0.26, -0.34, -0.30 (mean -0.30), for CBA, TSLA, LLY.

7. Reading the results

The one-edge KAN is, in effect, a spline, and behaves like one. Its error on CBA falls from 0.0707 (15 parameters) to 0.0131 (240), and at every budget from 60 upward it agrees with the classical least-squares spline to three or four decimals (CBA at 240: 0.0131 vs 0.0131; LLY at 240: 0.0083 vs 0.0081). The small gap at 15 and 30 parameters is bookkeeping: pykan spends nine of the budget on scales and fixed terms, so its spline has fewer intervals (6 against 12 at budget 15). It is deterministic to four decimals across seeds. Its rate, -0.59, is the roughness-limited N^(-1/2) rate.

Adding a hidden layer makes the KAN worse, at every budget, on every share. At 60 parameters on CBA the hidden-layer model's error is 0.0523 against 0.0272 for the one-edge model (1.9 times larger); at 240 it is 0.0388 against 0.0131 (3.0 times). Its rate of improvement is only -0.30: extra parameters buy very little. The fits (Figure 8) show why: the hidden-layer model draws the broad shape and misses the

medium-scale swings that the one-edge model captures.

Why depth hurts here. Each edge in the deep model is a spline on only 3 intervals: a curve with at most a couple of bends. Composing two such curves gives a curve with a few more bends, but still only a handful; it cannot manufacture the dozens of independent wiggles a price line has. The parameters go into more edges, not into resolution, and resolution is what a rough function needs. (This is a derivation from the architecture, consistent with the data; it is not a theorem.) The KAN paper's own successes with depth are on functions that have a genuine compositional structure, such as physics formulas, where composing simple pieces is exactly right.

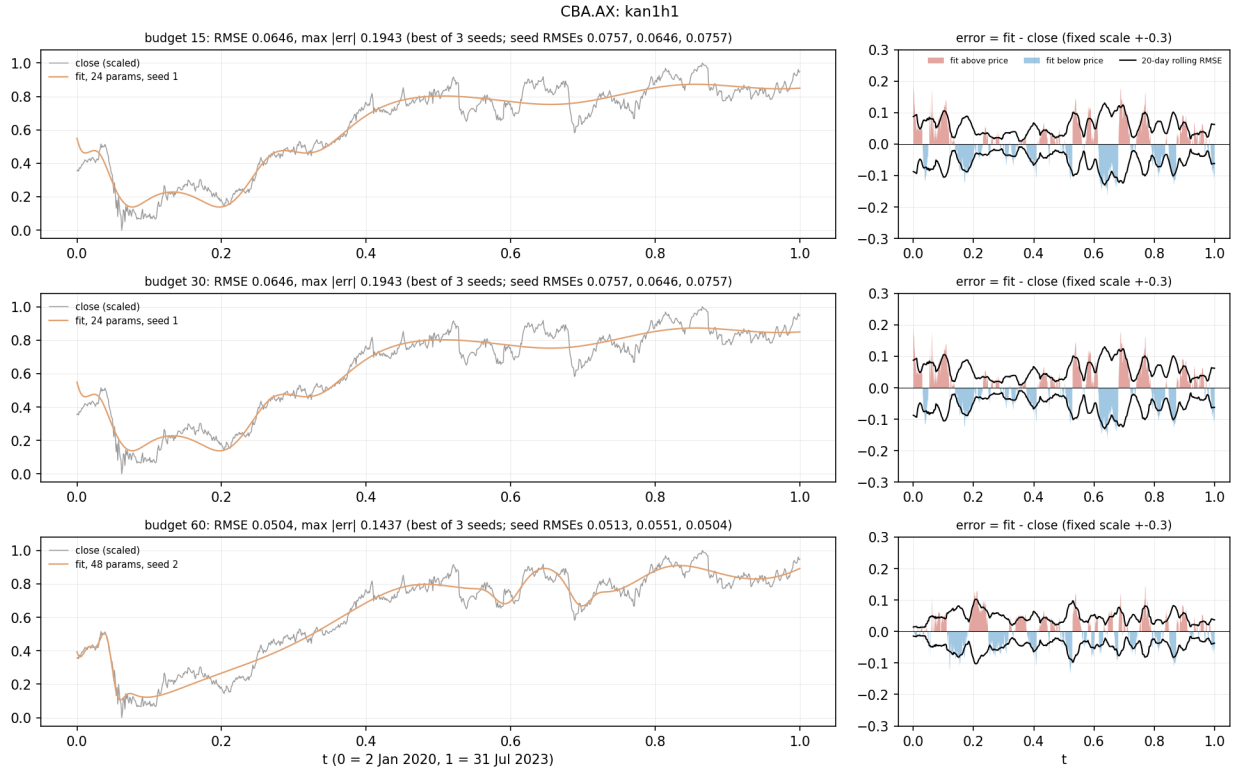


Figure 8. CBA, KAN [1,h,1] at 24, 24 and 48 parameters (h = 1, 1, 2): the fit (left) and the error wave with rolling-RMSE envelope (right). The envelope stays wide and the error has slow red/blue bands, meaning the model is missing whole swings, not just daily jitter. Compare with the one-edge panels in the result PDF.

8. What it means

A KAN with one edge is a good approximator of a price line because it *is* a spline; it inherits the spline's exact rate and has no advantage over it. The 'learnable activation function' that distinguishes KANs from ordinary networks is, on a single edge, precisely the classical object numerical analysts have used for sixty years, and the experiment confirms the identity numerically. Depth, which is where KANs are claimed to differ from splines, reduces accuracy for this task. If a case is to be made for KANs in describing price data, it cannot rest on accuracy per parameter; it would have to rest on properties not measured here, such as the readability of the fitted edge functions or the possibility of pruning them to a symbolic formula. And the error floor the one-edge KAN reaches is the N^(-1/2) roughness limit of the data, which no architecture can lower.

9. What this experiment cannot tell you (limitations)

• Everything is in-sample: the curve is scored on the very data it was fitted to. This is the right thing for an approximation study, but it says nothing about predicting prices that have not happened yet.

• Three shares and one period. The conclusions about the *models* are consistent across all three, but the numbers themselves would differ on other shares or periods.

• Parameter counts are matched approximately (within about 10 %), not exactly.

• Three random seeds; no formal significance test was applied. Where the spread across seeds is reported, treat differences smaller than the spread as not established.

• Only one grid size (G = 3) was used for the hidden-layer model; a deeper KAN with finer edge splines was not tried and might close part of the gap.

• pykan's optimiser is not bit-reproducible; differences of about 0.001 in RMSE between seeds are noise.

10. Glossary

term	meaning
Function	a rule turning an input (here, time t) into an output (here, a price).
Parameter	a number inside a formula that is chosen to make the formula fit the data.
Least squares	choosing the parameters so that the sum of squared errors is as small as possible.
RMSE	root-mean-square error: square the daily errors, average, take the square root; the typical error size.
Min-Max scaling	rescaling so the smallest value becomes 0 and the largest becomes 1.
Power law / rate	error proportional to N^(-a); a is the rate; on log-log axes a straight line of slope -a.
Holder exponent	a measure of roughness: how fast |f(x)-f(y)| shrinks as x and y approach; 1 = smooth, 1/2 = random-walk rough.
Seed	the starting value of the random number generator; different seeds give different random starts.
KAN	Kolmogorov-Arnold Network: a network whose edges carry learnable functions and whose nodes add.
Edge function phi	the learnable one-variable function on a KAN edge: a spline plus a silu ramp.
silu	the smooth ramp x / (1 + e^-x); pykan's fixed base term on every edge.
Grid G	the number of intervals in an edge's spline.
Hidden layer	a row of nodes between input and output; its size h sets the number of edges.
Composition	feeding the output of one function into another: psi(phi(t)).
L-BFGS	a numerical optimiser used to fit the KAN's parameters.